Multilingual speech to text options

Shared by Mohit

Speech to text multilingual is gemini 1.5 flash good enough or there are better, free or cheap versions available

Searched the web

Short answer: Gemini 1.5 Flash is outdated for this — swap it for Gemini 2.5 Pro/Flash or Whisper large-v3, both free/cheap options.

For multilingual speech-to-text right now:

Free/self-hosted:

- **Whisper large-v3** (OpenAI, MIT license) — still the gold standard for multilingual speech recognition, supporting 99+ languages and handling diverse acoustic environments and rare vocabulary effectively. Runs locally, zero API cost, just needs a GPU. Best pick if you want full control and no per-minute fees. Northflank
- **Faster-Whisper / WhisperX** — same model, optimized for speed.
- Newer open options like **NVIDIA Parakeet, Canary-Qwen, Moonshine** beat Whisper on raw WER benchmarks but are currently English-only — skip these if you need multilingual. Northflank

Cheap APIs:

- **Gemini 2.5 Pro/Flash** (not 1.5 — that's stale now) — for multilingual use, Gemini offers some of the broadest language coverage among hosted APIs, and it's cheap per-minute if you're already in the Google ecosystem or want multimodal context (video + audio together). Codesota
- **Deepgram Nova-3** — $0.0043/min batch, $0.0077/min streaming, sub-300ms latency, but weaker multilingual coverage than Whisper/Gemini — more of an English-first tool. NextLevel
- **AssemblyAI Universal** — good language coverage with automatic detection, competitive pricing.

Bottom line on accuracy vs. Gemini specifically: in recent side-by-side comparisons, Whisper still leads on finished-transcript quality and noise robustness, while Gemini is more competitive on speed, real-time streaming, and multimodal workflows — so if raw transcription accuracy is the priority, self-hosted Whisper large-v3 beats Gemini

and it's free. If you want low-latency + easy API + already touching Gemini/Vertex, go **Gemini 2.5 Flash**, not 1.5. DIY AI

Want a quick recommendation based on your specific use case (real-time vs. batch, languages involved, budget)?

> I am asking my indian user to send a voice not instead of clixking for example, 1 kg pyaaz, 1 kg dahi, 1 kilo ghee, sboudl be trnaslated in a manner that the ai learns and a json based strict parser sends me quantity, product name etc(backend deterministic) this is the use case

5:07 AM

Searched the web

This is a great use-case fit — you don't need generic multilingual STT here, you need something built for Hinglish/code-mixed grocery speech. Here's the architecture I'd use:

Step 1: STT — use Sarvam AI (Saaras v3), not Gemini/Whisper
Sarvam is an Indian company built specifically for this — their model was trained on 1M+ hours of real Indian audio and handles code-mixing (Hinglish/Tanglish) natively, at ₹1.5/min after free credits. Critically, it's a single unified model that automatically detects when a speaker switches between Hindi and English mid-sentence or drops in regional words, without losing accuracy at the language boundaries — this is exactly your "1 kg pyaaz, 1 kg dahi" pattern. Generic Whisper/Gemini will transcribe okay but tend to mangle colloquial food terms or normalize them oddly. Sarvam is cheap (~₹30/hour flat rate also listed) and purpose-built for this. Sarvam AI + 2

Step 2: Structured extraction — don't rely on the STT model for JSON
Keep STT and extraction as two separate steps for determinism:

1. **STT → raw transcript** ("1 kg pyaaz, 1 kg dahi, 1 kilo ghee")
2. **LLM extraction with strict JSON schema** (use Claude/Gemini with tool-use/function-calling, forcing schema like `{items: [{product, quantity, unit}]}`)
3. **Deterministic backend validation** — this is the important bit:
   - Maintain a canonical product dictionary (pyaaz→onion, dahi→curd, atta→flour, etc.)
   - Fuzzy-match the LLM's extracted `product` field against this dictionary (e.g. rapidfuzz) — reject/flag anything below a similarity threshold instead of trusting the LLM blindly
   - Normalize units (kg/kilo/किलो → "kg") via a fixed lookup table, not LLM judgement

- This way the LLM handles the messy language variation, but your backend stays deterministic and auditable

Why not skip straight to Gemini audio→JSON in one call? You can, and it's simpler, but it's harder to debug (can't inspect the intermediate transcript when something breaks) and less controllable for the product-dictionary matching step. For an order-taking system where wrong quantities have real cost, the two-step pipeline is safer.

Want me to sketch the JSON schema + fuzzy-matching pseudocode for this pipeline?

If I am going to maintaindictionary then whats the use of servam and ai

Whats gemini audio and for me at the start i want a free one because this proejct is just startung off, so which one shiuld we go for? Gemini 2.5 is it free? Gemi i audio? Sarvam?

Searched the web

Good questions — let me untangle both.

Why the dictionary matters even with Sarvam: Sarvam's job is just getting clean *text* out of messy *audio* — it turns "1 kg pyaaz" spoken audio into the string "1 kg pyaaz" reliably, even with code-switching. It doesn't know that "pyaaz" = onion in your product catalog, doesn't know your SKU IDs, and won't reliably output strict JSON on its own. The dictionary/fuzzy-match step is a separate job: mapping messy natural-language product names to your fixed catalog and catching spelling variants ("pyaz", "pyaaz", "pyaj"). Good STT reduces how often that matching step has to work hard — bad STT means garbage in, garbage out no matter how good your dictionary is. So it's STT (get accurate words) → LLM extraction (structure it) → dictionary/fuzzy-match (validate against your real catalog). All three layers do different jobs.

What "Gemini audio" is: It's just Gemini's native ability to accept audio files directly as input (not a separate product) — you send the voice note straight to the Gemini API and it can transcribe *and* reason over it in one call. Convenient, but less controllable than a dedicated STT step for your validation pipeline.

For starting free, here's the real picture right now:

Option	Free tier	Good for Hinglish?
Gemini 2.5 Flash / Flash-Lite	Yes — free via Google AI Studio, no card needed, ~1,500 requests/day, includes audio input	Decent, not specialized

Option	Free tier	Good for Hinglish?
Gemini 2.5 Pro	Free but capped hard at 50 requests/day	Better quality, too limited for testing volume
Sarvam Saaras v3	₹1,000 free credits on signup, then ~₹1.5/min	Best — purpose-built for exactly your Hinglish grocery-order case
Whisper (self-hosted)	Free forever if you have a GPU	Good but no code-mixing specialization

My actual recommendation to start: Use **Sarvam's free credits** first — it's specifically trained on Hinglish/code-mixed Indian speech, which is your exact use case, and ₹1,000 free credit will cover a lot of testing at ₹1.5/min. Once you've validated the pipeline, if cost becomes a concern at scale, benchmark Sarvam against **Gemini 2.5 Flash** (also free, good enough, but built for general multimodal reasoning, not tuned for Hinglish specifically) and pick whichever gives fewer parsing errors on real user audio.

Don't reach for Gemini Pro's free tier — 50 requests/day won't get you through a real testing cycle.

Whatsapp audio is where my chat bot is built

That changes a few practical details worth flagging:

Format: WhatsApp voice notes come as OGG/Opus (audio/ogg codec=opus). Both Sarvam and Gemini accept this, but double check:

- **Sarvam**: accepts common formats but you may need to confirm Opus support directly in their docs, or transcode to WAV/MP3 first (ffmpeg, one line, negligible cost) to be safe.
- **Gemini**: accepts audio/ogg natively as part of its multimodal input.

Pipeline via WhatsApp Cloud API (Meta's official API, assuming that's what you're using):

1. User sends voice note → Meta sends you a webhook with a `media_id`
2. You call Meta's `GET /{media_id}` to get a temporary download URL, then download the actual `.ogg` file (this is time-sensitive, URLs expire)
3. Send that audio file to Sarvam/Gemini for transcription
4. Pass transcript → LLM extraction → your dictionary/fuzzy-match validation

5. Return structured confirmation back to user via WhatsApp ("Got it: 1kg onion, 1kg curd, 1kg ghee — confirm?") before writing to backend — this catches STT/parsing errors before they become orders, which matters a lot for a grocery use case

One more thing worth deciding now: are you on the official **Meta WhatsApp Cloud API** directly, or going through a BSP/wrapper like Twilio, Gupshup, or a no-code tool (e.g., WATI, AiSensy)? That changes exactly how you fetch the media file and what auth/webhook shape you're dealing with — worth telling me so I can give you the exact download step.

Former

Good — direct Meta Cloud API gives you the most control. Here's the exact flow:

1. Receive the webhook
When a user sends a voice note, Meta POSTs to your webhook with a payload like:

```json
{
  "messages": [{
    "type": "audio",
    "audio": { "id": "MEDIA_ID", "mime_type": "audio/ogg; codecs=opus" }
  }]
}
```

Grab `audio.id`.

2. Fetch the media URL

```
GET https://graph.facebook.com/v21.0/{MEDIA_ID}
Authorization: Bearer {ACCESS_TOKEN}
```

Returns a temporary `url` (valid ~5 minutes, tied to your access token — you can't just curl it from a browser, needs the same bearer token).

3. Download the actual audio bytes

```
GET {url from step 2}
Authorization: Bearer {ACCESS_TOKEN}
```

Save as `.ogg` (Opus codec). Do this immediately — don't queue it, the URL expires fast.

4. Send to STT (Sarvam or Gemini)

- If Sarvam's API is picky about Opus, pipe through `ffmpeg -i input.ogg -ar 16000 output.wav` first (adds ~100ms, trivial cost, removes a failure mode).
- Otherwise send the `.ogg` directly.

5. Transcript → LLM JSON extraction → dictionary/fuzzy-match validation (as discussed).

6. Reply via Cloud API

```
POST https://graph.facebook.com/v21.0/{PHONE_NUMBER_ID}/messages
```

Send back a confirmation text ("1kg pyaaz, 1kg dahi, 1kg ghee — reply YES to confirm") before committing to your order backend — this is your safety net against STT/parsing errors, especially important early on while you're still tuning accuracy.

One gotcha to plan for: webhook processing must respond to Meta within a few seconds or it may retry/duplicate the event — so do the actual STT+LLM work asynchronously (queue it, e.g. via a simple job queue) and ack the webhook immediately, rather than blocking on the full pipeline inline.

Want the media_id → download → STT code sketch (Node/Python) to get you started?

Sarvam: accepts common formats but you may need to confirm Opus support directly in their docs, or transcode to WAV/MP3 first (ffmpeg, one line, negligible cost) to be safe.
Gemini: accepts audio/ogg natively as part of its multimodal input.

Does saram support ogg

Yes, Sarvam does support OGG audio input for speech-to-text, along with

Show more

Let me verify that directly against Sarvam's docs rather than assume it's accurate.

Searched the web

That's confirmed accurate — checked against Sarvam's own docs:

- **REST/Batch API**: Supported formats include WAV, MP3, AAC, AIFF, OGG, OPUS, FLAC, MP4/M4A, AMR, WMA, WebM, and PCM formats, with the API

automatically detecting most codec formats. So OGG/Opus works fine here. Sarvam

- **Streaming (WebSocket) API**: only supports WAV and raw PCM formats — other audio formats are not supported for real-time streaming. Sarvam

For your use case specifically: since WhatsApp voice notes arrive as complete files (not a live stream), you'll be using the **REST or Batch API**, not WebSocket streaming — so OGG/Opus is directly supported, no transcoding needed. You can skip the ffmpeg step entirely and just forward the downloaded `.ogg` file straight to Sarvam.

One practical note from their docs: the REST endpoint is best for responses under 30 seconds — grocery voice notes should easily fit that, but if you ever get longer rambling orders, route those to the Batch API instead. Sarvam

Explain how the costs will add up if an ona vwrage i get 100 users per hour using this voice note approach and sending on an average 15 second long audio note

Also i dodnt understand exolain 1.5 per minute how does that apply?

“1.5 per minute” just means you are billed 1.5 units of currency for every minute of processed audio (usually INR 1.5/min or similar, depending on

Show more

Let me verify the actual billing details before running your numbers — the pasted text has a pricing discrepancy I want to check (₹30/hour vs ₹1.5/min don't match, and that's a 3x difference that will change your math).

Searched the web

Good catch flagging that — the pasted numbers are inconsistent, and I want to give you real math, not guesses. Checked Sarvam's official pricing page directly:

Actual current rate: ₹45/hour for STT, **rounded up to the nearest second** — not the nearest minute. (Sarvam's own docs and marketing page disagree slightly, ₹30 vs ₹45/hour — I'm using the official pricing docs page as the authoritative number, but flagging you should double check your dashboard when you sign up, prices shift.)

Why "rounded to nearest second" matters for you specifically:
This is the key thing to understand. If billing were rounded up to the nearest *minute*, your 15-second clips would each be billed as a full 60 seconds — 4x more expensive than the actual audio length. Since it's rounded to the nearest *second* instead, you only

pay for the ~15 seconds you actually send. This is a big deal for a short-voice-note use case like yours.

Your actual cost math:

₹45/hour = ₹0.75/minute = ₹0.0125/second

- 1 voice note (15 sec) = 15 × ₹0.0125 = **₹0.1875** (~19 paise)
- 100 users/hour × 15 sec each = 1,500 seconds = 25 minutes of audio/hour
- Cost per hour = 25 × ₹0.75 = **₹18.75/hour**
- If that volume runs 24/7: ₹18.75 × 24 = **₹450/day** → **~₹13,500/month**
- If it's more realistic business hours (say 12 hrs/day): **~₹6,750/month**

Your free credits runway:
₹1,000 free credit ÷ ₹0.75/min ≈ 1,333 minutes of audio ≈ **~53 hours of usage at your 100 users/hour rate** before you'd need to pay anything. That's a solid amount of early testing/soft-launch room.

Don't forget the second cost layer: STT is just step one. The LLM JSON-extraction call (Gemini/Claude) on each transcript adds a small additional cost — but since your transcripts are tiny (a few words), this is usually a fraction of a paisa per request and won't meaningfully change the total. Text tokens are cheap; it's the audio processing that dominates your cost.

Can we use a free or open weight llm here for extraction instead of paying

Searched the web

Yes — and for your task specifically (short transcript → tiny JSON with product/qty/unit), this is actually a great fit for a small open-weight model, since it's a narrow, repetitive extraction job, not open-ended reasoning.

Best picks for your case:

- **Qwen3 (4B–8B)** — strong pick because it has strong multilingual support across 100+ languages and dialects, which matters since your transcripts will be Hinglish. Reliable structured/JSON output, Apache 2.0 license (no restrictions), runs comfortably on a single modest GPU or even CPU at small sizes.
- **Gemma 3 (4B)** — supports 140+ languages, similarly good multilingual coverage, and if you ever want to feed it the audio/image context directly it also has vision support.
- **Phi-4-mini (3.8B)** — the standout if you don't have a GPU at all — it runs on the machine you already have with no GPU required, good for early-stage prototyping

on a budget.

Reality check on quality: open-weight models still lag proprietary ones on non-English tasks specifically — in a recent Hindi/Telugu benchmark, Gemini 2.5 Flash was the strongest performer overall, and among open-weight models the gap between English and Hindi performance was larger than for closed models. So don't expect open-weight to match Gemini/Claude on nuanced Hindi understanding — but your task is narrow enough (extract product+qty+unit from a short sentence) that even a 4-8B model should handle it fine with good prompting, especially since you're also running fuzzy-match validation as a safety net afterward.

How to actually guarantee valid JSON (this matters more than model choice): don't just prompt-and-hope. Use **constrained decoding** — libraries like Outlines, or the XGrammar backend built into vLLM/SGLang — which masks invalid tokens during generation so the model is structurally incapable of producing malformed JSON, no retries needed. This is the difference between "usually works" and "always works," which you need for a deterministic backend.

Practical setup to start free:

- Run **Qwen3-4B or 8B** via **Ollama** (dead simple, one command) on a cheap GPU instance (or even a decent laptop for early testing) with vLLM + XGrammar once you scale
- Zero per-request cost, only your compute/hosting bill
- Given your STT cost is already the dominant expense (~₹19/hour at your volume), self-hosting extraction removes the second cost layer entirely and keeps your marginal cost per order near-zero

Want me to sketch the actual extraction prompt + JSON schema + Ollama/Outlines code to test this?

But I have to limit user to use a voice note, because of the billing

Smart move — capping usage protects your billing from runaway costs (spam, silence, background noise triggering re-sends, users rambling). Here's how to build that in cleanly:

1. Enforce a max duration before sending to STT (most important)
Check the audio duration locally with `ffprobe` right after downloading the file — before you pay for transcription:

bash

```
ffprobe -i voicenote.ogg -show_entries format=duration -v quiet -of csv="p=0"
```

If it's over your cap (say 30 seconds — matches Sarvam's REST API limit anyway, so a good natural cutoff), reject it and reply on WhatsApp: *"Voice note too long — please keep it under 30 seconds, just list your items."* This costs you nothing (no STT call made) and trains users toward short notes.

2. Rate-limit per user
Cap how many voice notes a single number can send per hour/day (e.g., 10/hour, 50/day). Store a counter in Redis or your DB keyed by phone number with a TTL. Once exceeded, reply: *"You've hit your order limit for now — try again in an hour"* or route to a human/text fallback. This stops spam or accidental duplicate sends from blowing up your bill.

3. Silence/junk detection before STT
Cheap pre-check: if the audio is mostly silence or below a volume threshold (quick check via `ffmpeg`'s `silencedetect` filter), skip the STT call entirely and ask the user to resend — no point paying to transcribe dead air.

4. Daily/monthly spend cap with alerts
Most providers (Sarvam included) let you set usage alerts on the dashboard. Set one well below your actual budget so you get warned before hitting free-credit exhaustion or an unexpected spike.

5. Dedup identical rapid-fire sends
If a user sends the same voice note twice in quick succession (common on flaky connections), hash the file or check timestamp proximity and skip re-processing the duplicate.

Rough sizing for your case: with the 30-second cap and rate limit, worst-case cost per user per hour is bounded and predictable — you can compute a hard ceiling (e.g., 10 notes/hour × 30 sec × ₹0.0125/sec = ₹3.75/user/hour max) rather than an open-ended bill.

Want the actual webhook code with the duration-check + rate-limit logic wired in?